

敏捷开发赋能项目式教学

一、敏捷开发的基本介绍

1.1 什么是敏捷开发？

敏捷开发是一种以人为核心、迭代式、渐进式的软件开发方法。它自 20 世纪 90 年代起逐渐引起广泛关注，目的是应对传统开发模式无法适应互联网，特别是移动互联网时代应用需求快速变更的挑战。在 20 世纪 90 年代之前的软件开发项目中，项目周期可以持续数年，同时因为用户需求来自专业领域（例如银行、航天等），开发团队可以深入调研用户需求、设计软件功能。这与现在主流的移动互联网时代的软件开发有截然不同的区别。现在的项目周期普遍以月为单位，要求能以周为单位、及时更新；同时，用户群体转变为更大规模的普通用户，软件需求由专业类的功能实现逐渐转变为日常类的体验，这导致软件需求多元化、更难以精准表达。

因此，与传统开发模式不同，敏捷并不追求在一开始就做出详尽设计或一次性编写完美代码，在敏捷过程中，软件项目会被拆分成多个较小的子目标。每个子目标在完成后都需要经过验证与测试，能够被集成并实际运行，从而逐步构建出完整系统。敏捷开发强调：

- 在较短的周期内交付具备核心功能的可用版本；
- 尽早让用户或客户使用并反馈；
- 在后续迭代中不断根据新需求或变化进行改进和完善。

简而言之，敏捷开发的重点是快速交付、持续改进、灵活响应变化，而不是“先设计到完美，再一次性交付”（图 1-1）。

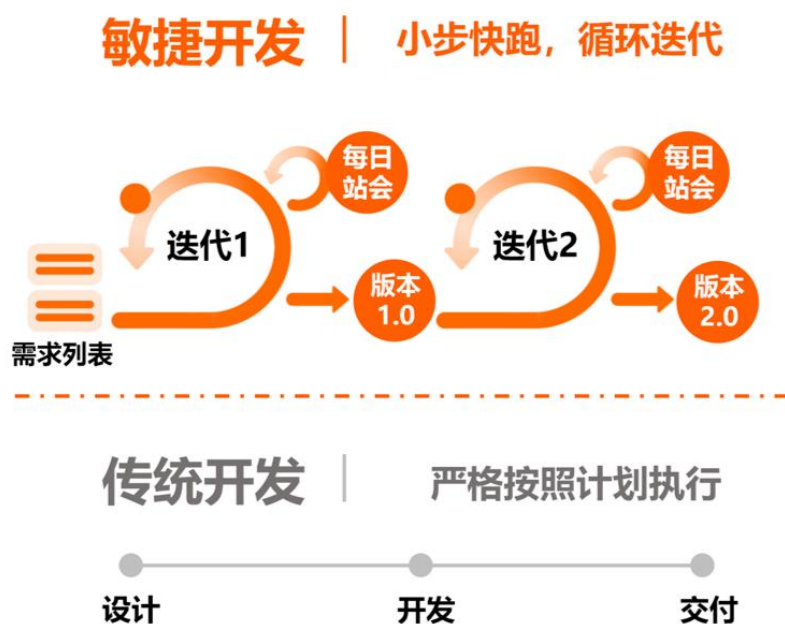


图 1-1 敏捷开发 VS 传统开发

1.2 敏捷开发的历史

20 世纪 90 年代，软件产业高速发展，但开发模式却面临瓶颈。那时主流的软件工程方法是“瀑布模型”：需求分析、设计、编码、测试、交付，层层递进。但实践中发现，这种模式在应对复杂多变的需求时存在严重问题：

- **需求变更困难：** 客户提出新需求时，往往要推翻原有设计，导致成本高昂。
- **交付周期过长：** 一年甚至数年的开发周期，用户迟迟看不到成果。
- **失败率高：** 由于中途缺乏反馈，往往到交付时才发现与用户期望偏差巨大。

在这种背景下，一些软件团队开始尝试更灵活的开发方式：

- 将任务分解成小模块，短周期交付；
- 更加重视与客户的沟通，而不是一味遵循合同；
- 注重团队协作和自组织，而不是严格的层级控制。

2001 年 2 月，17 位软件工程专家在美国犹他州的雪鸟（Snowbird）聚会，正式提出了《敏捷软件开发宣言》（Agile Manifesto）（图 1-2）。这份宣言只有短短几行，却成为全球软件开发的重要转折点。



图 1-2 敏捷软件开发宣言

从此，“敏捷”逐渐发展为一套方法论，并扩展到管理学、创新创业等多个领域。它的核心价值观和方法论，也给教学改革带来了新的启发。

1.3 敏捷开发的理念

《敏捷宣言》提出了四条价值观：

（1）个体和互动高于流程和工具

敏捷开发更注重团队成员的沟通与协作，而不是依赖严格的流程和工具。

山东某科技公司在早期过度依赖多种管理工具和繁琐流程，导致信息分散、沟通割裂、响应迟缓，成员逐渐形成“依赖工具传话”的习惯，协作效率大幅下降。为了解决这些问题，团队回归敏捷理念，精简工具体系，仅保留核心协作平台，并以每日站会和即时沟通替代冗长的书面流程，鼓励成员直接交流、快速决策。经过调整，项

目迭代周期从原先的 2~3 周缩短至 1~2 周，返工和误解减少约 40%，团队氛围与协作效率明显提升。这一案例表明，流程和工具只是辅助，真正推动项目高效运转的，是团队成员之间充分、及时、直接的互动。

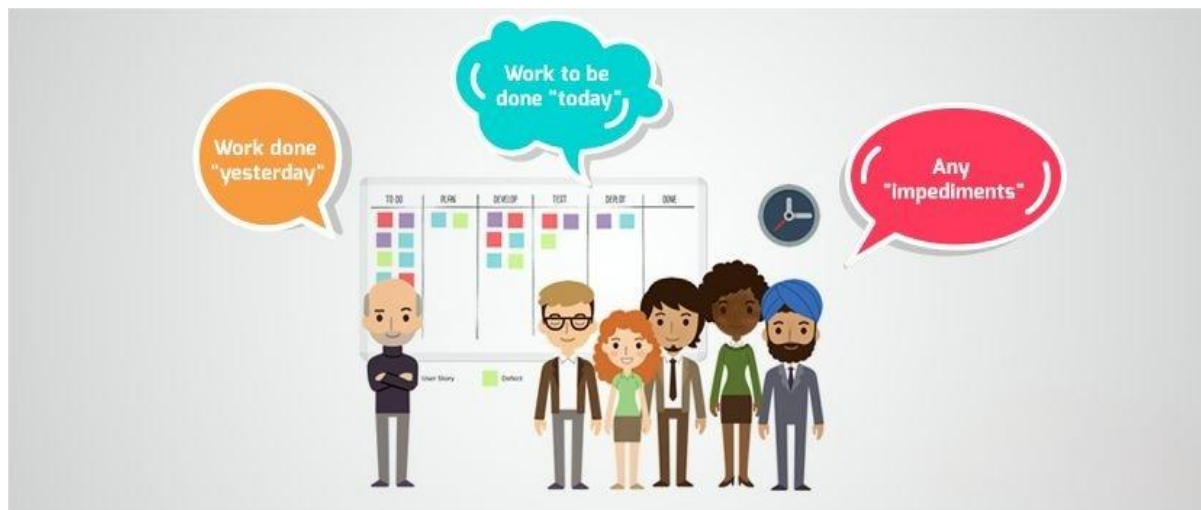


图 1-3 每日站会：即时沟通替代繁琐流程

（2）可工作的软件高于详尽的文档

敏捷开发更关注交付能使用的应用，而不是功能详尽的文档。文档应该服务于软件，而不是影响整个开发的进度。微信 APP 的发展就很好地印证了这一点。

微信从诞生起就遵循“先上线可使用的核心功能，再基于反馈优化”的原则，而非追求“完美文档再落地”。

2011 年微信 1.0 版本上线时，仅包含“即时文字聊天（图 1-4）、通讯录同步、头像设置”三个核心功能，既没有复杂的“功能规划文档”，也未涵盖“语音通话、朋友圈”等后续功能。

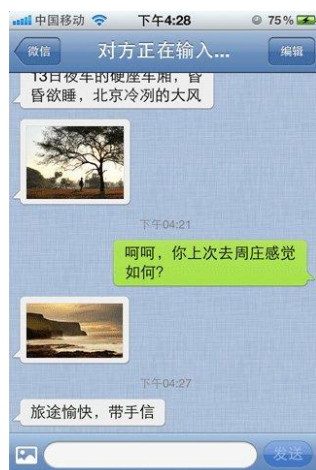


图 1-4 微信 1.0 即时文字聊天功能

但这个“极简但可工作”的版本，让团队快速验证了“移动端即时通讯”的可行性——通过用户反馈发现“文字输入效率低”，随即在 2.0 版本加入“语音消息”功能（图 1-5）。



图 1-5 微信 2.0 语音消息功能

若当时团队先花半年撰写《微信全功能规划文档》，涵盖所有设想的功能，不仅会错过“移动端社交崛起”的窗口期，还可能因文档中的“伪需求”（比如早期设想的“陌生人社交小游戏”）浪费资源，而“可工作的软件”则让迭代始终围绕用户真实需求推进。

(3) 客户合作高于合同谈判

敏捷开发强调与客户持续沟通和合作，而不是紧抓合同条款。小米 MIUI 系统从诞生起就践行“客户合作高于合同谈判”的理念，让用户深度参与迭代：

2010 年 MIUI 刚推出时，小米没按常规写“固定功能开发文档”，反而建论坛招募用户当“产品共创者”。工程师每天泡论坛收集反馈，新功能甚至由用户投票决定——比如 2014 年，超 10 万用户吐槽“垃圾短信太多”，小米没等“下版计划”，3 个月内就推出智能拦截功能；2020 年用户骂 K30 Pro 手机“没有高刷屏”，话题“#Redmi 还我高刷#”阅读破亿，小米 4 个月就推出搭载 120Hz 屏的升级版，价格不变，首销 1 分钟卖 3 亿。

更关键的是，这种合作不是“一次性反馈”，而是长期机制：每周“橙色星期五”更新版本，用户能直接给工程师提意见（图 1-7）；雷军每周看 TOP100 用户吐槽帖，连手机充电口防尘塞这种小细节，都是因用户抱怨“进灰”而连夜设计的。到 2024 年，小米用户提的 200 万条建议里，38% 都变成了实际功能，米粉复购率达 68%，远超行业水平。



图 1-6 小米四格体验报告

这里没有“固定合同”的束缚，用户不是“被动接受者”，而是和小米一起定义产品——这正是“客户合作高于合同谈判”的核心。

(4) 响应变化高于遵循计划

敏捷理念强调欢迎变化并快速调整计划，而非盲目固守原定安排，美团外卖 2022 年的骑手等餐优化正是这一价值观的体现。

2022 年前，美团外卖的配送流程遵循“算法工具预设”的原定计划——系统仅依据订单距离、商家历史出餐速度生成固定配送时间，骑手遇到等餐超时，只能按预设规则走申请流程补时，整个环节完全依赖工具逻辑，未考虑实际配送场景中的变量。

但随着骑手反馈增多，团队发现“九成骑手将等餐久列为最大痛点”，且超半数超时订单与等餐相关，这一突发的实际需求变化，打破了原有的工具流程计划。美团没有固守既定规则，而是快速响应：安排产品、算法人员与骑手面对面沟通，收集“商家出餐无预警”“到店不知优先等哪单”等真实场景反馈；随后紧急调整方案，推出“出餐宝”帮商家同步实时出餐状态，给骑手新增“建议到店时间”提示和一键报异常补时功能，同时向用户开放出餐进度查看，让流程适配实际需求（图 1-8）。

最终，骑手单均等餐时长下降 18%，用户相关客诉率降低 38%——通过及时响应骑手与用户的需求变化，打破原计划的工具流程束缚，既解决了核心痛点，也印证了“响应变化高于遵循计划”的价值。



图 1-7 美团取餐优化

除了价值观，《敏捷宣言》还提出了十二条原则，由于本书重点在教学，所以此处仅罗列跟教学比较相关的几条：

- 频繁交付可用成果 → 学生应定期展示学习产出；
- 欢迎需求变化 → 教学设计要允许根据学生实际情况调整；
- 激励个体 → 学习动机和自主性是教学成功的关键；
- 面对面交流 → 有效沟通比文件交流更高效；
- 团队自组织 → 学生小组需要具备一定的自治权；
- 定期反思改进 → 课程应安排“回顾环节”，促进自我迭代。

简而言之，敏捷开发的理念强调：动态适应、持续改进、团队协作、快速交付。这与教育中“以学生发展为中心”的理念高度契合。

1.4 敏捷开发的基本方法

敏捷开发绝非停留在口头的管理口号，而是一套经过数十年行业验证、覆盖“目标设定 - 过程管理 - 质量保障 - 成果交付”全链路的可操作方法论体系。它的核心价值在于打破传统“瀑布式开发”中“需求固化、周期冗长、反馈滞后”的痛点，通过“小步快跑、快速迭代、持续反馈”的模式，让团队能灵活响应需求变化，同时确保交付成果的实用性与高质量。以下将以两大核心方法（Scrum、看板）展开深度解析，补充关键细节、扩展实践场景与行业案例，帮助更全面地理解其落地逻辑。

（1）Scrum（最流行的框架）

在介绍 Scrum 之前，我们先了解几个常见的基本概念：

- 冲刺周期（Sprint）：可以理解为一个阶段性的“小目标”。一般为 2 - 6 周，团队需要在这个时间内完成并交付明确的成果。

- 用户故事（User Story）：用户提出的具体需求，用通俗的话描述。比如在微信里，一个“用户故事”可能就是“我希望能用语音输入”或“我希望能发朋友圈”。
- 任务（Task）：将“用户故事”进一步拆解成可以分配给个人的小任务。
- 需求清单（Backlog）：团队待完成的需求列表，分为产品需求总清单（Product Backlog）和某个冲刺周期的任务清单（Sprint Backlog）。
- 每日站会（Daily Meeting）：每天十几分钟的小型交流会，团队成员快速同步进展和困难。
- 评审会议（Sprint Review Meeting）：一个周期结束后，团队向大家展示成果。
- 燃尽图（Sprint Burn Down Chart）：记录和展示当前周期里任务的完成情况，直观反映进度。
- 版本发布（Release）：一个或多个周期完成后，交付新的可用版本。

Scrum 的运作过程大致如下：



图 1-8 Scrum 工作流程

除了基础定义，每个概念背后都隐含着“规避风险、提升效率”的设计逻辑：

- Sprint（冲刺周期）

每个冲刺周期包含 2-6 周。这个周期并非随意设定 —— 过短（如 1 周）会导致“任务拆解过度、会议成本升高”，过长（如 8 周）则会丧失敏捷“快速反馈”的优势。

多数互联网团队会选择 2 周为一个 Sprint，既能保证交付有价值的成果，又能及时调整方向。例如电商团队在“618 大促”前，会以 2 周为周期，依次完成“商品报名功能”“优惠券系统”“支付链路测试”等冲刺目标。

● User Story（用户故事）

它的核心不是“记录需求”，而是“以用户视角定义价值”，通常遵循“3C 原则”——Card（卡片，简洁记录）、Conversation（对话，团队与用户对齐需求）、Confirmation（确认，明确验收标准）。

比如外卖 APP 的一个用户故事不会写“开发订单备注功能”，而是写“作为用户，我希望在下单时添加备注（如‘不要香菜’），这样能吃到符合口味的餐品”，后者清晰传递了“用户价值”，避免团队陷入“为了开发而开发”。

● Task（任务）

拆解用户故事时需满足“INVEST 原则”——Independent（独立）、Negotiable（可协商）、Valuable（有价值）、Estimable（可估算）、Small（足够小）、Testable（可测试）。

例如“用户语音输入”的用户故事，可拆解为“设计语音录制界面”“开发语音转文字接口”“测试不同环境下的语音识别准确率”等任务，每个任务需明确负责人、预估工时（如 4 小时 / 8 小时），确保可落地。

● Backlog（需求清单）

Product Backlog（产品需求总清单）由产品负责人（Product Owner）维护，需持续“排序”（Prioritization），常用“RICE 模型”（Reach 影响范围、Impact 影响程度、Confidence 信心、Effort 所需 effort）判断优先级。

例如教育 APP 中，“修复课程播放卡顿”（高影响、低投入）会排在“新增个性化学习报告”（高价值、高投入）之前。而 Sprint Backlog 是从 Product Backlog 中挑选的“当前冲刺可完成的任务”，一旦确定，冲刺期间原则上不新增任务（除非出现严重风险），避免团队精力分散。

● 每日站会 (Daily Meeting)

严格控制在 15 分钟内，核心是“同步进度、暴露问题”，而非“讨论解决方案”。团队成员需依次回答三个问题：“昨天完成了什么？”“今天要做什么？”“遇到了什么阻碍？”。

例如开发工程师可能会说：“昨天完成了登录接口联调，今天要做用户信息页面开发，目前遇到的问题是第三方头像接口返回延迟，需要后端协助排查”——此时 Scrum Master（敏捷教练）会记录问题，会后协调资源解决，避免站会变成“技术研讨会”。

● 评审会议 (Sprint Review Meeting)

冲刺结束后，团队需向“利益相关者”（如产品负责人、客户、测试团队）展示“可工作的产品增量”（Potentially Shippable Increment），而非“PPT 演示”。

例如工具类 APP 团队在评审会上，会直接演示“新上线的截图翻译功能”，让参会者实际操作，收集反馈（如“翻译速度太慢”“支持的语言太少”），这些反馈会被纳入下一个 Sprint 的 Product Backlog。

● 燃尽图 (Sprint Burn Down Chart)

纵轴是“剩余任务工时”，横轴是“冲刺天数”，理想状态下曲线应“逐步下降”，直至冲刺结束时归零。若曲线出现“平台期”（如连续 2 天剩余工时不变），说明团队可能遇到了阻塞（如技术难题、需求理解偏差），Scrum Master 需及时介入。燃尽图的成功使用需要对工作量有清晰、准确的评估。

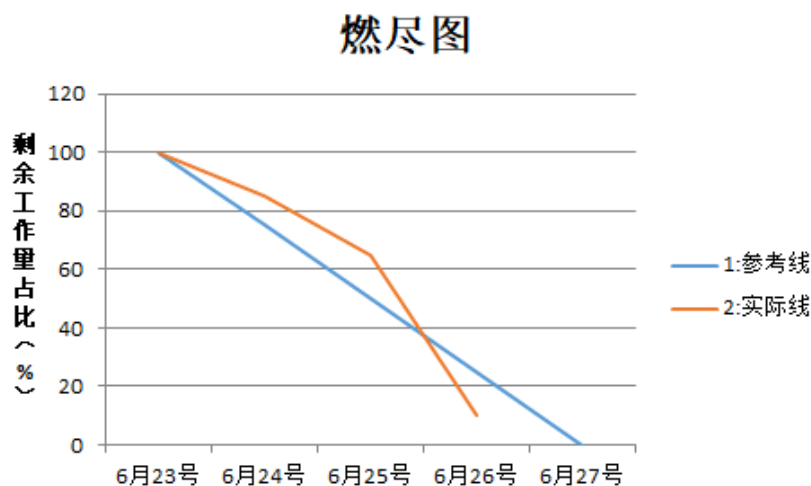


图 1-9 燃尽图示例

例如某团队的燃尽图在冲刺第 5 天出现平台期，排查后发现是 “设计图未按时交付”，随即协调设计团队优先输出，避免影响整体进度。

● 版本发布 (Release)

并非每个 Sprint 都必须发布版本，需结合业务需求 —— 例如 To B 软件（如企业 ERP 系统）可能 3-4 个 Sprint 后发布一个大版本，而 To C 软件（如社交 APP）可能每个 Sprint 结束后发布小版本（如修复 bug、优化体验）。发布前需经过 “验收测试”（UAT），确保产品增量符合用户故事的验收标准，避免 “带病上线”。

①Scrum 完整运作流程：从启动到迭代的闭环

a. 项目启动阶段：明确核心目标

产品负责人 (PO) 首先组织 “需求研讨会”，与客户、业务团队对齐项目核心价值（如 “开发一款面向大学生的求职 APP，核心目标是 ‘帮助用户 30 分钟内完成简历制作’ ”），然后梳理出初始的 Product Backlog，并用 “MoSCoW 方法”（Must have 必须有、Should have 应该有、Could have 可以有、Won't have 暂不有）标记需求优先级，确保团队先聚焦 “必须实现的功能”。

b. Sprint 规划会议 (Sprint Planning)

每个冲刺开始前，团队召开规划会，时长建议为 2~4 小时，分为两步：

第一步：PO 向团队讲解 “本次冲刺要交付的核心价值”（如 “完成简历模板选择与编辑功能”），并确认团队对需求的理解。

第二步：团队将选中的用户故事拆解为 Task，估算每个 Task 的工时（常用 “故事点” Story Point，如 1/2/3/5/8，而非具体小时，避免精确但不准确的估算），并承诺 “本次冲刺能完成的故事点总量”（即 “速度” Velocity）。例如团队估算 “简历模板选择”（2 个故事点）、“简历编辑功能”（5 个故事点）、“简历预览功能”（3 个故事点），总故事点为 10，若团队历史速度为 8-12，说明目标合理。

c. Sprint 执行阶段：聚焦交付

冲刺期间，团队按计划推进任务，通过每日站会同步进度，用燃尽图跟踪工时消耗。Scrum Master 需扮演 “移除障碍者” 的角色 —— 例如开发团队遇到 “服务器资源不足”，Scrum Master 需协调运维团队扩容；测试团队反馈 “需求文档模糊”，Scrum Master 需组织 PO 与测试团队补充细节，避免团队因外部障碍停滞。

d. Sprint 收尾阶段：交付与改进

冲刺结束后，先召开评审会，收集相关者反馈；再召开回顾会，确定改进措施；最后 PO 确认 “产品增量是否符合验收标准”，若符合，则将其纳入 “已完成的工作”（Done），并更新 Product Backlog（如将评审会收集的 “简历导出 PDF 功能” 加入下一个 Sprint 的待选需求）。

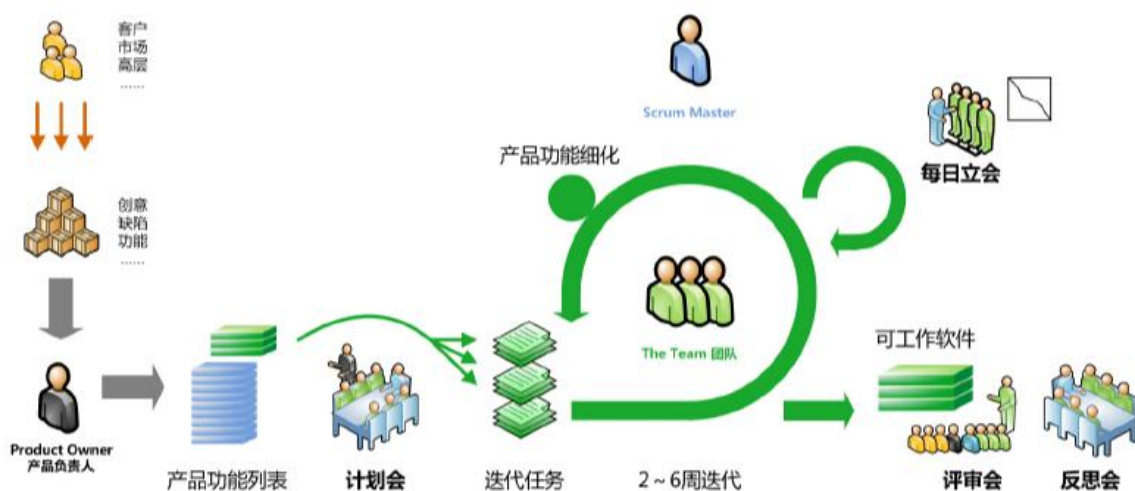


图 1-10 Scrum 敏捷开发流程

②Scrum 的行业实践案例

- **腾讯微信团队：**在“视频号”功能迭代中，采用 2 周 Sprint 周期——第一个 Sprint 完成“视频上传与播放”，第二个 Sprint 实现“点赞、评论功能”，第三个 Sprint 上线“直播预约”，通过每轮评审会收集用户反馈（如“播放卡顿”“评论区交互不流畅”），快速调整下一轮优先级，最终让视频号在短时间内实现功能完善。
- **微软 Azure 团队：**作为 To B 云服务团队，采用 4 周 Sprint 周期，每个 Sprint 结束后向企业客户交付“功能预览版”，通过客户的实际使用反馈（如“云存储扩容速度慢”“安全权限配置复杂”）优化产品，确保正式发布时能满足企业客户的核心需求。

（2）看板（Kanban）

如果说 Scrum 是“有节奏的分段冲刺”，看板（Kanban）则是“灵活的持续流动”——它源于日本丰田的“精益生产”（Just-In-Time），核心是通过“可视化任务流”，减少“任务阻塞”，提升团队协作效率，尤其适合需求频繁变化、无需严格周期的场景。

它的做法很简单：把任务写在卡片上，贴在“待办、进行中、已完成”三个栏目里（图 1-11），任务完成一点就往右挪一点。久而久之，整个团队的工作一目了然。

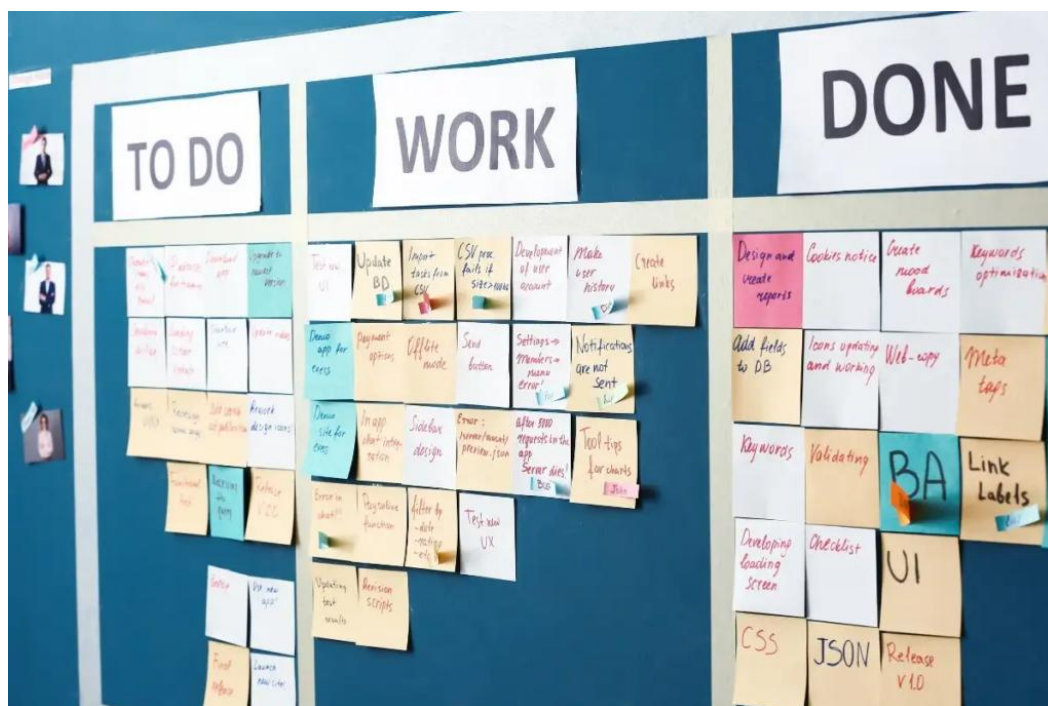


图 1-11 看板项目管理

1) 看板的核心原理与设计逻辑

看板的本质是“限制在进行中的工作数量（Work In Progress, WIP），让任务快速流动”，而非简单的“贴卡片”。其核心设计需遵循三大原则：

- **可视化流程：**将任务从“启动”到“完成”的全流程拆解为多个“阶段”，每个阶段对应一个“看板列”，确保每个任务的状态清晰可见。除了基础的“待办（To Do）、进行中（In Progress）、已完成（Done）”，复杂项目可增加细分列，例如软件开发的看板可设计为：“需求评审 → 设计 → 开发 → 测试 → 上线 → 验收”，每个列代表一个明确的工作阶段。
- **限制在进行中的工作数量（WIP Limit）：**这是看板区别于“传统任务列表”的关键——每个“进行中”阶段（如“开发”“测试”）需设定最大任务数量，避免团队同时处理过多任务导致效率下降。例如“开发列”设定 WIP=3，意味着团队最多同时进行 3 个开发任务，若有新任务需进入开发列，必须等其中一个任务完成并流转到“测试列”后才能加入。这种限制能迫使团队先解决“阻塞任务”（如某个开发任务因接口问题卡住），而非不断新增任务。

- **持续优化流动：**通过跟踪“任务周期时间（Cycle Time）”（从任务进入“进行中”到“完成”的时间），发现流程中的瓶颈。例如某团队发现“测试列”的任务平均停留 5 天（远高于开发列的 2 天），说明测试资源不足或测试流程繁琐，此时团队可调整——如增加测试人员、优化自动化测试脚本，减少任务在测试阶段的阻塞。

2) 看板的落地场景与工具

看板的灵活性使其不仅适用于软件开发，还能渗透到各行各业，甚至个人任务管理：

- **互联网公司的跨团队协作：**阿里巴巴的电商运营团队用看板管理“大促活动筹备”，看板列设计为“活动方案评审 → 商品招商 → 营销素材制作 → 活动上线准备 → 活动复盘”，每个任务卡片标注“负责人、截止时间、依赖项”（如“营销素材制作”依赖“商品图拍摄”）。团队成员通过在线工具（如 Jira、飞书多维表格）实时移动卡片，例如“商品招商”完成后，卡片从“商品招商”列移到“营销素材制作”列，素材团队看到后立即启动工作，无需反复沟通“该谁接手”。
- **传统企业的生产管理：**某汽车零部件工厂用物理看板管理“零件生产流程”，每个生产环节（如“原材料切割 → 零件加工 → 质量检测 → 包装”）对应一块看板区域，每个零件附带一张“生产卡片”，完成一个环节后，工人将卡片移到下一个区域，管理人员通过看板直观看到“哪个环节堆积了零件”（如“质量检测”区域卡片过多，说明检测效率低），及时调整人员配置。
- **学生社团的活动组织：**大学学生会用看板管理“迎新晚会筹备”，看板列分为“节目征集 → 节目筛选 → 舞台设计 → 宣传推广 → 晚会执行”，每个节目对应一张卡片，标注“负责人、节目类型、排练进度”。社团成员每天查看看板，例如“节目筛选”完成后，卡片移到“舞台设计”列，设计组立即根据节目需求（如“舞蹈节目需要 LED 背景”）推进工作，避免“信息差”导致的延误。
- **个人任务管理：**职场人可用 Notion 或 Trello 搭建个人看板，列设计为“待办（今天）、进行中、待反馈、已完成”，将“写周报”“对接客户需求”“学习

新工具”等任务填入卡片，完成一项就向右移动，既能避免任务遗漏，又能直观看到“当前有多少任务在等待反馈”（如“对接客户需求”卡在“待反馈”列），及时跟进推进。

2) 适用场景与选择建议

很多团队会纠结“选 Scrum 还是看板”，核心是看“需求稳定性”和“团队协作模式”，两者的关键差异可通过下表进一步细化：

表 1-1 Scrum 与看板的对比

	Scrum	看板
需求处理 方式	需求需提前规划，冲刺期间原则上 不变更	需求可随时加入，支持动态调整优先 级
周期与节 奏	固定周期（2-6 周），按周期交付	无固定周期，任务完成后即可交付 （持续交付）
角色分工	明确角色（PO、Scrum Master、开 发团队），各司其职	无固定角色，可根据需求灵活分配 （如“需求提出者”“任务执行 者”“验收者”），复杂项目可引入 敏捷教练
核心指标	速度（Velocity）：每个冲刺完成的 用户故事总量，用于预测后续进 度	周期时间（Cycle Time）：任务从启 动到完成的平均时间； 在进行中的工作数量（WIP）：控制 流程阻塞的关键指标
适用场景	需求相对稳定、需明确周期交付的 项目（如开发一款新 APP）	需求频繁变化、需快速响应的场景 （如电商运营活动、客户支持工单处 理）

团队协作 依赖	依赖标准化会议（每日站会、评审会）同步信息	依赖可视化看板自动同步信息，减少会议成本
------------	-----------------------	----------------------

①选择 Scrum：当你需要 “明确的目标感” 和 “结构化流程”

Scrum 适合 “需求有初步优先级、需要阶段性交付成果” 的场景，比如：

- 互联网产品的版本规划（如 “V1.0 做核心功能，V1.1 加社交分享，每 3 周一个版本”）；
- 企业级项目的落地（如 “给客户开发 CRM 系统，每 4 周交付一个模块，让客户确认进度”）；
- 跨部门协作项目（如 “市场部、技术部、设计部合作做活动页，需要固定节奏同步进度”）。

不适合：需求完全不确定（无法拆解 Sprint 目标）、团队人数过少（如 2-3 人，角色分工反而增加成本）。

②选择 Kanban：当你需要 “灵活响应” 和 “可视化进度”

Kanban 适合 “需求动态变化、任务类型多样、无需固定交付周期” 的场景，比如：

- 运维团队的日常工作（如 “处理服务器故障、响应业务部门的配置需求、升级系统”，任务随机到来，需优先处理紧急项）；
- 内容创作团队（如 “公众号文章撰写：选题→初稿→编辑→排版→发布，任务持续流动，无固定周期”）；
- 学生社团的活动筹备（如 “校园晚会筹备：节目征集→场地申请→宣传物料→彩排，任务可随时新增，进度需全员可见”）。

不适合：需要严格控制交付时间（如 “必须在月底前上线”，Kanban 无固定节奏易拖延）、团队成员对任务优先级判断不一致（易导致 “抢活” 或 “漏活”）。

在实际工作中，很多团队不会“纯用某一个框架”，而是根据需求融合不同框架的优势，常见的融合模式有：

● Scrum + Kanban（“Scrumban”）：结构化与灵活性结合

很多互联网团队会用 Scrum 的“固定冲刺周期”设定目标，同时用 Kanban 的“看板”可视化 Sprint 内的任务进度——比如：

- 每个 Sprint 开始时，产品负责人确定冲刺目标，团队将任务拆解后写到 Kanban 看板的“待办”列；
- 日常工作中，团队成员从“待办”列领取任务，完成后移到“进行中”“测试中”“已完成”列，同时用看板的 WIP 限制（如“进行中”列最多放 3 个任务）避免成员同时处理过多任务；
- 冲刺结束后，按 Scrum 流程开评审会和回顾会，同时通过 Kanban 的“周期时间”分析“哪个环节耗时最长”（如“测试环节平均耗时 2 天，需优化”）。

这种模式既保留了 Scrum 的“目标感”，又通过 Kanban 提升了任务透明度，适合“需求有优先级但冲刺内可能微调”的项目。

（5）总结：敏捷开发的核心不是“选框架”，而是“对齐目标”

无论是 Scrum，还是 Kanban，本质都是“解决问题的工具”——选择的关键不是“哪个框架更先进”，而是“哪个框架（或融合模式）能更好地解决当前项目的痛点”：

- 如果你需要“明确的阶段性成果”，用 Scrum；
- 如果你需要“灵活应对动态任务”，用 Kanban；

敏捷开发的最终目标，是让团队在“不确定的环境中”高效交付有价值的成果——框架是手段，不是目的。只要能够实现“小步迭代、快速反馈、持续改进”，哪怕是简化后的框架，也是成功的敏捷实践

二、敏捷开发在教学中的实践

2.1 为什么教学需要敏捷？

随着新一代信息技术的快速发展和数字经济的深度演进，高等教育正面临前所未有的挑战与变革。一方面，人工智能、大数据、云计算、区块链、元宇宙等前沿技术不断涌现，推动产业结构和社会形态持续快速变化；另一方面，学生的知识结构、学习方式、职业发展路径日益多元，人才培养目标也逐步从“知识传授”转向“能力培养”“创新引领”和“终身发展”。传统以学科为中心、线性化、刚性化的教学体系，已经越来越难以适应这一时代背景下对高素质创新人才的培养要求。教学体系亟需具备更强的灵活性与适应性——这正是“敏捷教育”提出的现实基础和时代必然。

早在 2019 年，徐晓飞教授在《计算机教育与可持续竞争力》蓝皮书中，就提出了“敏捷教学”（Agile Education）的概念，强调以学生发展为中心，通过理论、技术、实践的交叉并行与快速重构，构建高度灵活、动态适应的人才培养体系，培养具有持续竞争力的创新型人才。该理念与《ACM/IEEE CC2020》报告中提出的“胜任力（Competence）”目标相呼应，强调在多变环境中学生的应变力与创新力培养。

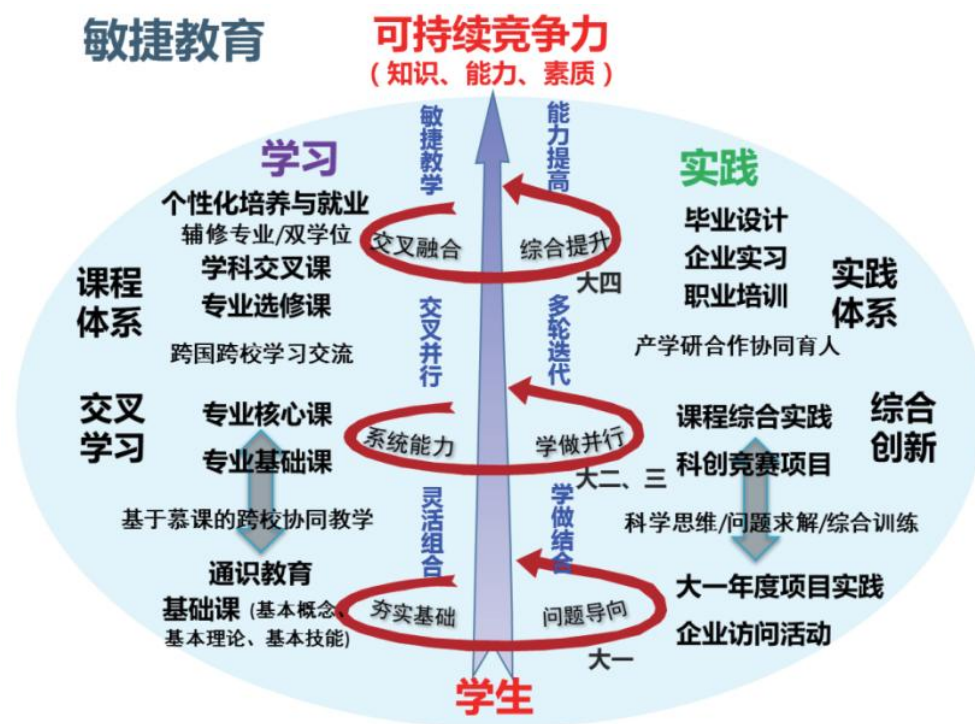


图 2-1 面向可持续竞争力的敏捷教育模式 [来源：《计算机教育与可持续竞争力》]

哈工大威海校区自 2017 年以来便率先将敏捷教育理念融入新工科建设，围绕“ Π 型”人才培养目标，通过跨学科融合、产学合作与国际联合培养，创新构建了新工科“8 Π 模型”，并建立矩阵式未来技术学院，推动课程体系、教学方式、管理体制等全面重构。在此模式下，通识教育与专业教育、理论与项目学习、校内与企业实践、国内与国际培养深度融合，支撑学生在快速变化的技术与产业环境中持续成长。

与此同时，国外顶尖高校也在积极探索敏捷化的教学模式。以斯坦福大学为代表，其在《Stanford 2025》计划中提出了“开环大学”（Open Loop University）的理念，打破传统四年制刚性培养模式，为学生提供在校与社会之间多次自由进出的学习机会，强调学习的灵活性与适应性。麻省理工学院（MIT）通过跨学科教学实验室和项目制学习，将真实工程问题引入课堂，快速迭代课程内容以匹配产业技术发展；芬兰阿尔托大学则以“设计思维+项目驱动+跨学科团队”为核心，推动学生在多轮实践与反思中形成创新能力与复杂问题解决能力。这些探索都体现出：面对未来，教育体系必须具备“敏捷性”，才能持续响应技术进步、产业变革与个体需求。



图 2-2 智能化、开放式、服务型教育服务生态体系

敏捷教育不是对传统教学的简单补充，而是一种面向未来的教育体系重构。它通过目标分类、动态重构、灵活组合、学做结合、交叉并行与多轮迭代，打破知识、学科、机构的边界，使教学体系能够快速响应技术与社会发展的变化，满足学生多样化成长路径的需要。敏捷教育的引入，将为我国高等教育培养具有可持续竞争力的复合型、创新型人才提供有力支撑。

2.2 敏捷开发与教育理念的契合

敏捷开发的核心理念——**价值优先、持续迭代、快速反馈、团队协作**——与成果导向教育（Outcome-Based Education, OBE）和项目制教学的基本要求高度契合（见图 2-2）。它不仅能够为 OBE 提供切实可行的实施方法，还能为项目制教学提供高效的过程组织与管理框架，从而使教学活动在动态变化的环境中保持灵活性与高效性，帮助学生真正实现“学以致用”的能力目标。

OBE 强调“以产出为导向”，即所有教学目标、教学过程与评价方式，都应围绕学生最终能够达到的学习成果来设计。相较于传统的“以教师讲授为中心”的模式，OBE 更关注学生是否真正掌握了解决实际问题的能力；而项目制教学强调以真实或拟真的项目为载体，让学生在任务驱动、团队协作和持续迭代中完成学习过程。敏捷开发方法与两者的理念高度契合，在教育实践中具有天然的结合基础和广泛的应用潜力。

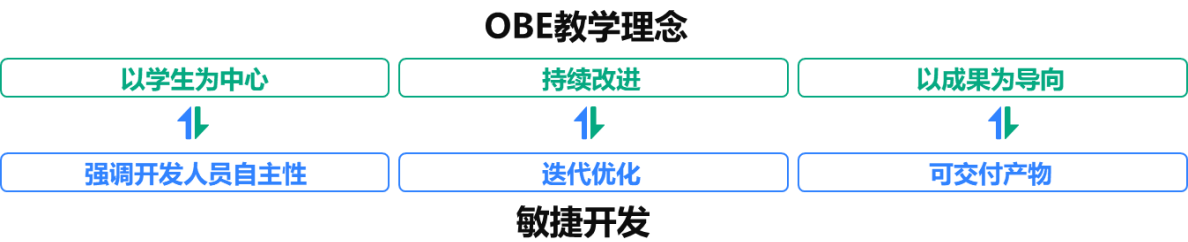


图 2-3 OBE 理念与敏捷开发

2.2.1 与 OBE 的契合

(1) 以学生为中心 对应 团队协作 + 自驱动学习

OBE 倡导以学生为核心，鼓励学生在学习中主动去探索、积极参与，教师主要起引导和支持作用；敏捷开发依靠跨职能团队的紧密协作，团队里每个人都要主动担责、贡献力量。二者结合能充分激发学生的参与感与责任感，学生不再是被动接受知识，而是像敏捷团队成员一样，基于自驱动去学习，教师则如同 Scrum Master，更多扮演“促进者”的角色。

（2）持续改进 对应 迭代优化 + 冲刺回顾

OBE 认为学习是一个持续尝试、反思与修正的过程；敏捷开发则通过短周期迭代（Sprint）和冲刺回顾（Sprint Retrospective）来不断优化产出。在教学中，通过阶段性汇报、小组讨论或课堂“站会”等形式，学生可以像敏捷团队一样及时获取反馈、调整策略，使学习过程保持动态改进，避免“期末一交了之”的单一评估方式。

（3）以成果为导向 对应 用户故事 + 可交付产物

OBE 要求教学目标必须清晰指向学生能力的实际成长，强调“会用、能做”；敏捷开发中，每个迭代都必须设定明确的交付目标，以可交付产物回应用户需求。在教学场景下，可通过“用户故事”明确学生的学习目标，并在各阶段形成与能力培养直接对应的学习成果或作品，使学生的产出可见、可评、可用。

敏捷开发不仅是软件工程的方法论，更是一种可以与 OBE 理念相辅相成的教育方法。它既能为 OBE 提供具体可行的操作手段，也能让 OBE 的产出导向在课堂中更好地落地。通过二者的结合，教学能够在动态变化的环境中保持灵活性与前瞻性，为学生未来的职业发展打下坚实基础。

2.2.2 与项目制教学的契合

项目制教学（Project-Based Learning, PBL）强调以真实或贴近真实的项目任务为核心，让学生在解决复杂问题的过程中实现知识与技能的综合应用。其关键特征包括任务驱动、真实情境、团队协作、阶段性成果与反思总结。而这些特征与敏捷开发方法的核心原则高度一致，二者结合能够形成更高效、更具实效性的教学实施路径。

（1）项目目标与敏捷迭代目标天然契合

项目制教学通常需要将整体任务分解为多个阶段性子目标，对应敏捷开发中的多轮迭代（Sprint）。每个迭代聚焦明确的教学目标与交付产出，学生能够在逐步推进项目的过程中实现能力的循序渐进式提升。

（2）项目团队组织与敏捷团队结构相互呼应

项目制教学强调团队分工与协作，敏捷开发通过跨职能团队与日常站会、回顾会等机制，帮助团队保持高效沟通与持续改进。将敏捷机制引入教学，可有效解决学生团队在项目推进中常见的沟通不畅、进度失控、责任不清等问题。

（3）快速反馈机制强化教学质量保障

传统项目制教学往往侧重期末成果展示，过程反馈不足；敏捷开发则强调通过短周期评审、用户反馈和团队反思，持续优化产出。应用于教学中，教师和同伴可以在每个迭代结束后进行成果评审和学习反思，使教学过程透明、可调整，提升整体教学质量。在敏捷框架下，教师不再是单一的知识传递者，而更像项目指导者与团队“教练”，通过机制设计与过程干预，帮助学生高效完成项目并培养自主协作与问题解决能力。

将敏捷开发方法与项目制教学结合，不仅能够提升教学的组织效率和过程质量，还能更好地培养学生在真实情境下的综合应用与团队协作能力，为其未来走向产业实践奠定坚实基础。

2.3 教学目标

在敏捷开发理念适配的项目制教学中，教学目标不再局限于“完成项目任务”或“掌握知识点”，而是通过敏捷实践构建一套多维目标体系——既关注学生能力的深层培养，也重视教学过程的高效运转，更强调教学成果的实际价值。这一目标体系能帮助教师清晰把握“敏捷赋能教学”的核心方向，确保教学改革不流于形式。

（1）学生能力目标：从“被动执行”到“主动创造”